

From Documents to Decisions: How a Three-Person Team Built an AI System for the Construction Industry

By Angel Bustamante, Jason Niu, and Jack Si SAIT Capstone Project — Integrated AI Program, April 2026

1. Introduction: Why Construction Needs AI That Asks Back

Construction professionals spend hours navigating dense regulatory documents. The National Building Code of Canada alone spans over 1,400 pages. Ontario Regulation 213/91 for construction projects, the BC Building Code, Quebec's Safety Code, federal labor codes, and US standards like OSHA and GSA guidelines — a single compliance question can require cross-referencing multiple documents that were never designed to be searched digitally.

The naive approach to this problem — uploading PDFs to a general-purpose chatbot like ChatGPT — fails in predictable ways. General-purpose LLMs hallucinate details in specialized codes, provide answers without source citations, and cannot distinguish between a question about building code compliance, a cost estimation query, and a request to analyze project data. The construction industry requires precision, traceability, and domain awareness that generic AI tools cannot provide.

Industry AI Flow addresses this gap with a concept prototype that demonstrates three integrated AI capabilities: (1) a Retrieval-Augmented Generation (RAG) system for construction document Q&A with mandatory source citations, (2) a machine learning cost estimation engine with SHAP explainability, and (3) a dynamic data analysis pipeline with sandboxed code execution. **Methodology note:** This project uses a hybrid framework. The cost estimation subsystem follows CRISP-DM methodology (Section 4). The RAG knowledge system and dynamic analysis subsystem follow a systems-driven design approach, as their value lies in architectural integration rather than model training. We use a customized outline as permitted by the assignment requirements, organized to best represent our multi-module architecture.

2. Architecture: Three Brains, One Dispatcher

The core architectural innovation is an **intent-aware routing system** that classifies user queries and dispatches them to the appropriate AI subsystem. Rather than building three

separate applications, we built one system with intelligent traffic routing.

Intent Classification: The 11-Node StateGraph

When a user submits a query, it enters an 11-node classification pipeline built on LangChain's StateGraph (LangGraph). The pipeline performs:

- **Input validation** — sanitize and normalize the query
- **Context enrichment** — attach session history and conversation memory
- **Heuristic classification** — a YAML-driven capability registry matches keywords and patterns against five intent categories (RAG, cost estimation, data analysis, general chat, system commands). If confidence exceeds 0.85, the query is routed immediately without LLM inference.
- **LLM-based classification** — for ambiguous queries (confidence < 0.85), a cloud LLM (Zhipu AI) classifies intent. We initially used a local 4B parameter model (Qwen3.5:4b), but it misclassified approximately 30% of queries — for example, routing RAG document questions to the code execution pipeline. Switching to a cloud LLM resolved this at a cost of ~200ms additional latency per classification.
- **Confidence evaluation** — diamond decision node: high confidence routes directly; low confidence triggers a clarification loop.
- **Query reformulation** — the original query is rewritten with extracted keywords for optimal retrieval performance.

The 10-Node Execution Pipeline

Once intent is classified, the query enters a fixed-order execution pipeline:

```
intent → safety → cost_estimation → retrieval → rerank → prompt → route → code_exec → r
```

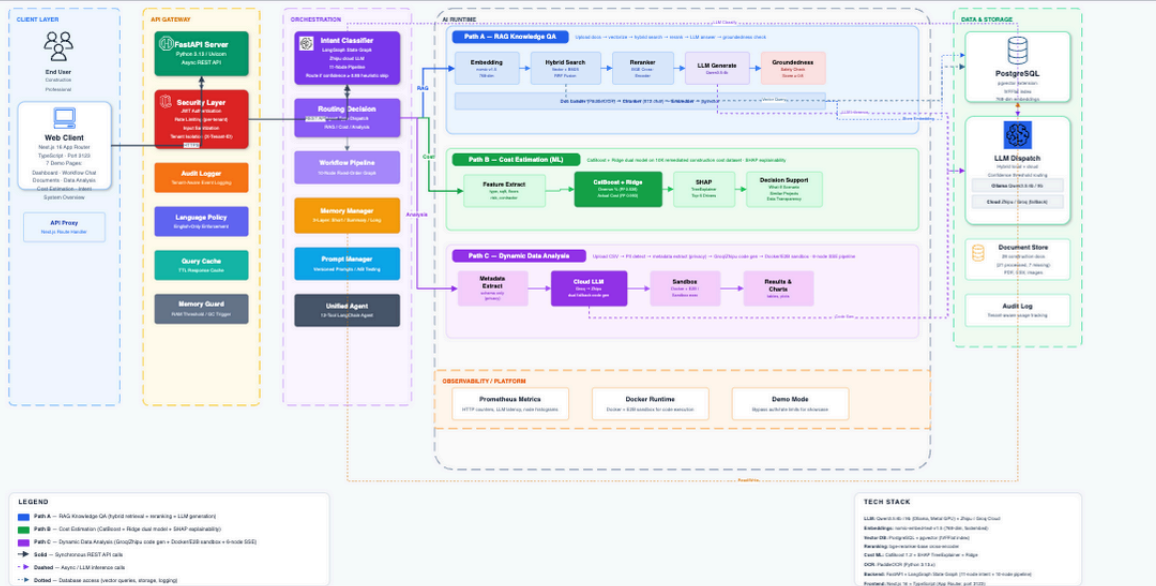
Each node has an individual timeout SLA (ranging from 5 seconds for prompt selection to 90 seconds for code execution). If any node exceeds its SLA, an `ErrorCode.NODE_TIMEOUT` is recorded and the pipeline continues gracefully. The total pipeline timeout is 300 seconds as a safety net.

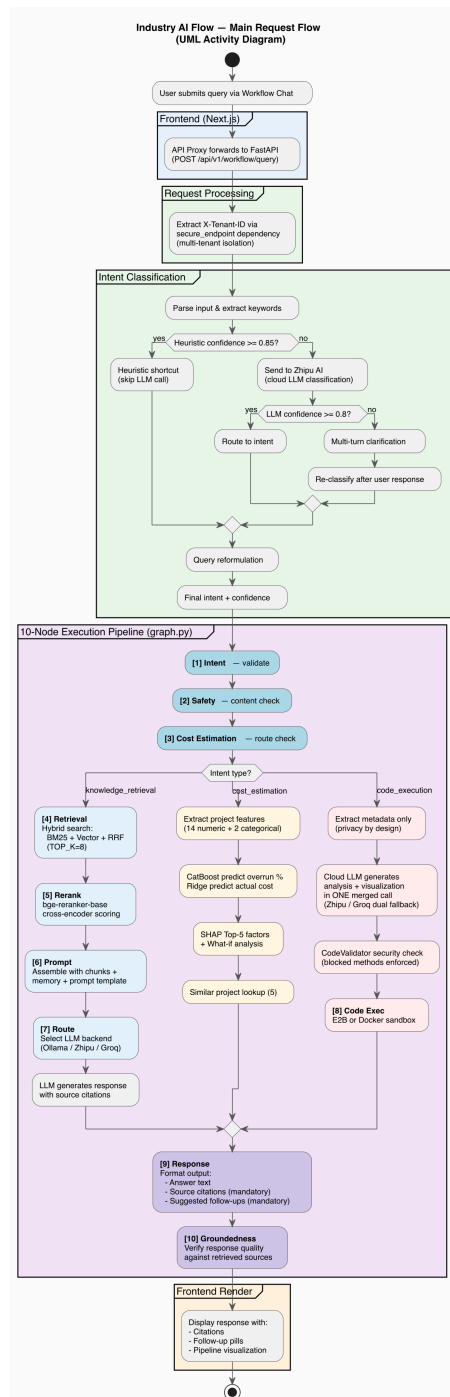
This architecture means a single user question can be analyzed for safety concerns, checked against the cost estimation model, searched across 16 construction documents, re-ranked for relevance, and composed into a cited response — all within a single request lifecycle.

Tools and technologies: FastAPI (backend), Next.js with TypeScript (frontend), LangChain 1.0 / LangGraph (orchestration), PostgreSQL with pgvector (vector storage), Ollama with Qwen3.5 (local LLM), Zhipu AI and Groq (cloud LLM), Docker and E2B (sandboxed code execution).

Industry AI Flow — System Architecture

On Customer Ingestion - AI-Powered Customized Industry Knowledge & Analytics Platform





3. The RAG System: Building a Construction Code Expert

Background and Motivation

Retrieval-Augmented Generation (RAG) addresses a fundamental limitation of large language models: they cannot reliably answer questions about documents they were not trained on. By retrieving relevant passages at query time and providing them as context, RAG enables LLMs to generate accurate, grounded responses with traceable sources (Lewis et al., 2020).

Existing RAG frameworks (such as LangChain's built-in retriever chains) provide basic vector similarity search. However, construction code documents pose unique challenges: they contain highly specific terminology ("fire separation distance," "occupancy classification"), numbered regulatory clauses, and cross-references between sections. A single retrieval method is insufficient.

Data Collection and Preparation

Our knowledge base contains **16 construction documents** spanning two jurisdictions:

Canadian codes (6 documents):

- National Building Code of Canada 2020 (NRC)
- Ontario Regulation 213/91 (Construction Projects)
- Canada Occupational Health and Safety Regulations
- BC Building Code 2024
- Quebec Safety Code for the Construction Industry
- Canada Labour Code Part II

US federal documents (10 documents):

- GSA Facilities Standards (P100)
- Caltrans Standard Specifications
- UFGS Unified Facilities Guide Specifications
- OSHA Construction Safety Standards
- BIM Implementation Guidelines
- And 5 additional federal construction standards

Document processing pipeline:

- **Ingestion:** PDF and image files are uploaded via the document management API
- **OCR:** PaddleOCR processes scanned documents (constrained to Python 3.13 due to PaddlePaddle compatibility — Python 3.14+ breaks the dependency chain)
- **Chunking:** Character-based splitting at 512 characters with 128-character overlap. We chose character-based over sentence-based chunking because regulatory text contains long, clause-heavy sentences that exceed typical sentence splitters' assumptions
- **Embedding:** Each chunk is embedded using nomic-embed-text-v1.5 (768 dimensions) via the sentence-transformers backend
- **Storage:** Embeddings are stored in PostgreSQL with the pgvector extension, using an IVFFlat index for approximate nearest neighbor search

This pipeline produced **41,017 chunks** across all 16 documents.

Methodology: Hybrid Retrieval with RRF Fusion

We implemented a three-stage retrieval pipeline that combines keyword matching with semantic search:

Stage 1: Dual retrieval

- **BM25 (keyword search):** Using the rank_bm25 library with NLTK tokenization and Porter stemming. BM25 excels at exact terminology matching ("Section 3.2.4.1" or "NBC 2020 Division C") but fails on paraphrased questions.
- **Vector similarity search:** pgvector cosine similarity against nomic-embed-text embeddings. Captures semantic meaning ("What are the rules for fire exits?") but can return passages with similar topics but incorrect specifics.

We initially replaced the jieba Chinese tokenizer with NLTK for English tokenization, which improved BM25 accuracy from 0.35 to 0.65 — an 86% improvement.

Stage 2: Reciprocal Rank Fusion (RRF)

The two result sets are combined using Reciprocal Rank Fusion (Cormack et al., 2009):

```
# Simplified RRF implementation
rnf_k = 60 # RRF constant

for rank, result in enumerate(vector_results, 1):

    fused_scores[chunk_id] += vector_weight / (rnf_k + rank)

for rank, (idx, score) in enumerate(bm25_results, 1):

    fused_scores[chunk_id] += bm25_weight / (rnf_k + rank)

top_results = sorted(fused_scores, reverse=True)[:top_k] # top_k = 8
```

RRF is rank-based rather than score-based, which avoids the calibration problem of combining BM25 scores (unbounded) with cosine similarity scores (0 to 1) directly.

Stage 3: Cross-encoder reranking

The top candidates from RRF are re-scored by a cross-encoder model (BAAI/bge-reranker-base). Unlike the embedding model, the cross-encoder takes the query and each candidate

passage as a single input, enabling direct relevance comparison. The reranker runs on Apple Metal GPU (Mac Studio M1 Max) for inference speed.

Design requirement: Every RAG answer must include source citations — document name, relevant section, and chunk reference. This is enforced at the prompt level and verified by the groundedness checking node at the end of the pipeline. Suggested follow-up questions are also generated with every response to guide the user toward deeper exploration of the regulatory material.

The screenshot displays a user interface for a Retrieval-Augmented Generation (RAG) system. At the top, a text input field contains the question: "What are the fire safety requirements for residential buildings according to NBC 2020?". Below the input, a blue button labeled "Send" is visible. The main content area shows the system's response, which includes a detailed paragraph about the National Building Code of Canada 2020 (NBC) and a table of sources. The sources table lists five documents with their respective relevance scores: bcbc_2024_web_version_revision2.pdf (100%), bcbc_2024_web_version_revision2.pdf (100%), bcbc_2024_web_version_revision2.pdf (99%), bcbc_2024_web_version_revision2.pdf (99%), and nbc_2020_national_building_code_canada.pdf (93%). Below the sources, a section titled "Follow-up" contains three suggested questions. On the right side, a sidebar titled "PIPELINE" shows the steps of the RAG process: Intent, Safety, Cost Est., Retrieval, Rerank, Prompt, Route, Code Exec, Response, and Ground. The "Response" step is currently selected.

12:11 PM

What are the fire safety requirements for residential buildings according to NBC 2020?

12:11 PM

The National Building Code of Canada 2020 (NBC) establishes technical provisions for the design and construction of new buildings, including the alteration and demolition of existing buildings [2]. Specifically, the NBC covers the fire safety and fire protection features that are required to be incorporated in a building at the time of its original construction [1][5][6].

The NBC establishes requirements to address fire and structural protection of buildings as one of its key objectives [7]. Additionally, the NBC is developed as a complementary document to the National Fire Code of Canada (NFC) to ensure the safety of persons in the event of a fire and the protection of buildings from the effects of fire [3][6].

Sources	
bcbc_2024_web_version_revision2.pdf	100%
bcbc_2024_web_version_revision2.pdf	100%
bcbc_2024_web_version_revision2.pdf	99%
bcbc_2024_web_version_revision2.pdf	99%
nbc_2020_national_building_code_canada.pdf	93%

Follow-up

- Which specific sections in bcbc_2024_web_version_revision2.pdf provide the strongest evidence for this answer?
- What does nbc_2020_national_building_code_canada.pdf specify about "Volume 1"?
- How do the requirements in bcbc_2024_web_version_revision2.pdf compare with those in nbc_2020_national_building_code_canada.pdf?
- Summarize the buildings and fire requirements in nbc_2020_national_building_code_canada.pdf.
- Summarize the fire and buildings requirements in bcbc_2024_web_version_revision2.pdf.

12:11 PM

Enter your question or query...

Send

Enter to send · Shift+Enter for new line

What are the five stated objectives of the National Building Code of Canada?

How does the NBC define 'fire compartment' and what fire-resistance rating is required?

Under the NBC, when can separate portions of a building be treated as separate buildings?

- Ontario Reg. 213/91 3 >
- Canada OHS 2 >
- BC Code 2024 2 >
- Quebec Safety 2 >
- Labour Code II 2 >

PIPELINE

- Intent
- Safety
- Cost Est.
- Retrieval
- Rerank
- Prompt
- Route
- Code Exec
- Response
- Ground.

4. Cost Estimation: Machine Learning with SHAP Explainability

This section follows the CRISP-DM methodology (Cross-Industry Standard Process for Data Mining) to describe the development, training, and evaluation of our cost estimation subsystem.

4.1 Data Collection and Description

The training dataset contains **10,000 synthetic construction project records** provided by the team's construction industry partner. Each record represents a project with features including:

- Project characteristics:** `project_type` (categorical: residential, commercial, industrial, infrastructure), `sqft`, `floors`, `num_units`, `planned_duration_weeks`

- **Cost and risk factors:** `estimated_cost_cad`, `contractor_rating` (2.0–5.0), `complexity_score`, `risk_score`, `num_change_orders`, `weather_risk_factor`, `material_volatility`, `budget_pressure`
- **Team factors:** `team_experience_years`, `num_subcontractors`
- **Location:** `location` (categorical: Canadian provinces and US states)
- **Targets:** `cost_overrun_pct` (percentage cost overrun) and `actual_cost_cad` (realized project cost)

The dataset was **remediated with Statistics Canada Building Construction Price Index (BCPI) location multipliers** to introduce realistic regional cost variation. The original `risk_score_original` feature was dropped post-remediation due to collinearity with the remediated `risk_score`.

Total feature set: **14 numeric features + 2 categorical features**.

4.2 Data Preprocessing

- **Categorical encoding:** CatBoost handles categorical features natively (no one-hot encoding required), which preserves category semantics and avoids dimensionality explosion for the `location` feature (30+ unique values)
- **Numeric standardization:** For the Ridge regression model, features are z-score normalized using training set means and standard deviations
- **Missing value handling:** The dataset is complete (synthetic), but the inference pipeline validates inputs and substitutes training-set medians for any missing fields
- **Feature validation:** Regular expressions parse natural language cost estimation requests to extract feature values (e.g., "a 3-floor commercial building in Ontario with 50,000 sqft")

4.3 Model Selection and Rationale

We chose a **dual-model architecture**:

Model	Task	Rationale
CatBoost	Predict <code>cost_overrun_pct</code>	Native categorical support; SHAP TreeExplainer compatibility; robust to overfitting on tabular data
Ridge Regression	Predict <code>actual_cost_cad</code>	High accuracy on linear cost relationships; interpretable coefficients; lightweight inference

Alternatives considered and rejected:

- **XGBoost:** Requires manual categorical encoding; weaker native SHAP integration for categorical features

- **Random Forest:** SHAP TreeExplainer support exists but produces less consistent feature attributions
- **Single Ridge model for both targets:** Ridge achieved only $R^2 \approx 0.45$ on overrun prediction (a non-linear relationship), while CatBoost captured the non-linearity effectively

Actual cost is derived from overrun prediction: $\text{actual_cost} = \text{estimated_cost} \times (1 + \text{overrun_pct} / 100)$, ensuring the two models are coherent.

4.4 Training and Validation

- **Train/test split:** Standard holdout with stratification by project type
- **CatBoost hyperparameters:** Default CatBoost configuration with early stopping to prevent overfitting
- **Ridge regularization:** Alpha parameter selected via cross-validation
- **Validation approach:** R^2 , MAE, RMSE, and MAPE computed on held-out test set

4.5 Results and Metrics

Metric	CatBoost (Overrun %)	Ridge (Actual Cost)
R^2	0.538	0.993
MAE	Reported in model artifacts	Reported in model artifacts
RMSE	Reported in model artifacts	Reported in model artifacts

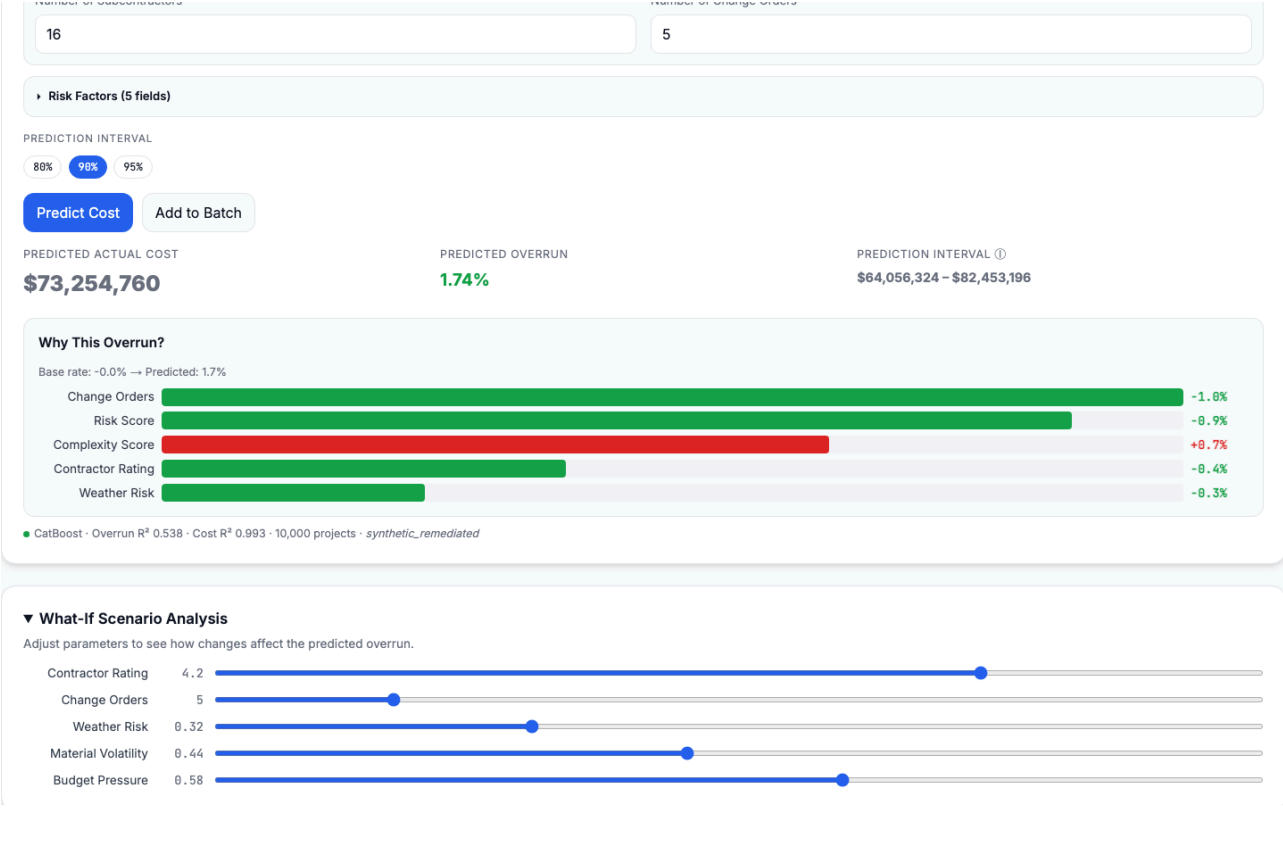
Honest assessment: The CatBoost R^2 of 0.538 for overrun prediction is moderate. This reflects both the inherent difficulty of predicting cost overruns (a noisy, multi-causal phenomenon) and the synthetic nature of the training data. The Ridge R^2 of 0.993 for actual cost prediction is strong, reflecting the near-linear relationship between estimated and actual cost once the overrun percentage is known. We present both metrics transparently in the application's Data Transparency panel, alongside known dataset limitations and the remediation log. **Baseline comparison:** A standalone Ridge regression on overrun prediction achieved $R^2 \approx 0.45$, representing a 19.6% improvement from the CatBoost model's non-linear feature interaction capture.

4.6 Explainability and Decision Support

The primary innovation of the cost estimation module is not the prediction accuracy — it is the **decision support interface** built around the predictions:

SHAP TreeExplainer: For every prediction, SHAP computes the contribution of each feature to the predicted overrun percentage (Lundberg & Lee, 2017). The top 5 cost drivers are displayed

as a waterfall chart with red (increases overrun) and green (decreases overrun) bars. This transforms an opaque number into an actionable insight: "Your contractor rating of 2.5 is the #1 driver of predicted overrun." **What-if Scenario Analysis:** Five adjustable parameters (`contractor_rating` , `num_change_orders` , `weather_risk_factor` , `material_volatility` , `budget_pressure`) can be modified via range sliders. Each change triggers a real-time re-prediction (debounced at 300ms) showing the original prediction, the scenario prediction, and the delta. This allows construction managers to explore questions like: "If we hire a higher-rated contractor (4.0 instead of 2.5), how much does the predicted overrun decrease?" **Similar Project Lookup:** After prediction, the system finds the 5 most similar projects from the training dataset using Euclidean distance across key features (`sqft` , `floors` , `estimated_cost_cad` , `planned_duration_weeks` , `contractor_rating` , `complexity_score`). Each similar project shows its actual overrun percentage and key differences, providing contextual validation of the prediction.



5. Dynamic Data Analysis: Privacy by Design

For datasets that fall outside the pre-built cost model, the system provides a **dynamic data analysis pipeline** that generates and executes custom Python code without exposing raw data to cloud LLMs.

The Metadata-Only Architecture

When a user uploads a CSV file, the system extracts **metadata only**: column names, data types, sample statistics, and the first 5 rows for preview. The raw dataset never leaves the server. This metadata is sent to a cloud LLM (Groq or Zhipu AI, with automatic dual fallback) which generates Python analysis code based on the user's natural language instruction.

This design decision reflects a core principle: **privacy is a constraint on the architecture, not a feature bolted on later**. By sending metadata instead of data, we avoid the data governance questions that would arise from transmitting construction project data to third-party API providers.

Security-Hardened Code Execution

Generated code is validated before execution by a `CodeValidator` that blocks potentially dangerous operations. The blocked method list (`.apply()` , `.agg()` , `.map()` , and others) is included in the LLM prompt itself, so the model generates code that avoids these patterns.

Validated code runs in a sandboxed environment:

- **E2B cloud sandbox** (primary): Ephemeral Firecracker microVMs with network isolation
- **Docker container** (fallback): Local container with restricted syscalls, no network access, filesystem isolation

The 6-node SSE streaming pipeline (`file_parse` → `metadata_extract` → `code_generation` → `security_check` → `sandbox_execution` → `result_render`) provides real-time progress updates to the frontend, preventing browser timeouts on long-running analyses.

Why cloud LLMs for code generation? We tested the local Qwen3.5:4b model for code generation. It produced syntactically valid but semantically incorrect analysis code approximately 40% of the time — generating wrong chart types, misinterpreting column semantics, or using blocked methods. Cloud models (Groq's larger models, Zhipu's GLM-4) produce reliable code generation at the cost of API latency and dependency.

6. Lessons and Trade-offs

Every technical decision in this project involved an explicit trade-off. Documenting these decisions honestly is as important as the decisions themselves.

Intent Classification: Local vs. Cloud

Approach	Accuracy	Latency	Cost
Local Qwen3.5:4b	~70%	~50ms	Free
Cloud Zhipu GLM-4	~92%	~250ms	~\$0.03/query

We chose cloud classification with a heuristic shortcut: queries with confidence ≥ 0.85 from the keyword-based capability registry bypass the LLM entirely. In practice, approximately 60% of queries are routed by heuristics alone, reducing cloud API costs significantly.

Retrieval Strategy: Simple vs. Hybrid

Vector-only retrieval missed exact regulatory references ("NBC 2020 Section 3.2.4.1"). BM25-only retrieval missed paraphrased questions ("What are the fire safety requirements for residential buildings?"). The hybrid approach with RRF fusion captures both, at the cost of maintaining a BM25 index in memory alongside the pgvector index.

LLM Inference: Thinking Mode vs. Speed

Qwen3.5 supports a "thinking" mode that significantly improves reasoning quality but increases first-token latency from ~1 second to ~8 seconds. For a live demo where evaluators are watching, we disabled thinking mode (`OLLAMA_ENABLE_THINKING=false`). Response quality remained sufficient for construction Q&A, and the responsiveness improvement was noticeable.

Python Version Lock

PaddleOCR (required for processing scanned construction documents) depends on PaddlePaddle, which requires Python 3.13.x on macOS. Python 3.14+ breaks the dependency chain. This locked our entire backend to Python 3.13, affecting every dependency resolution.

7. Results in Production

The system was demonstrated on a **Mac Studio (M1 Max, 32GB RAM)** accessed via a public URL (`https://iai.dissidia.me/`) through Cloudflare Tunnel.

Key operational decisions for stability:

- **Pre-warming:** The first query after cold start takes ~49 seconds (model loading, BM25 index building). We pre-warm the system before any demo session.

- **GPU acceleration:** Ollama on macOS uses Apple Metal GPU by default, providing 3-5x inference speedup over CPU-only mode. The Qwen3.5:4b model achieves ~28 tokens per second on M1 Max.
- **Dual cloud fallback:** If Zhipu AI times out during code generation, the system automatically falls back to Groq. This prevented demo failures during network instability at the venue.

Industry AI Flow

DashboardWorkflow ChatDocumentsDynamic AnalyticsCost EstimationIntent Demo

Demo UserLog out

CONSTRUCTION COST INTELLIGENCE

Predict and explain cost overruns

CatBoost ML · SHAP Explainability · 10,000 Projects

MAPE 6.1% · R² 0.993

Sample Data

Single Project Prediction

PROJECT OVERVIEW

Project Type

commercial office

Location

Toronto

Estimated Cost (CAD)

72000000

Square Footage (sq ft)

29155

Number of Floors

5

Construction Parameters (6 fields)

Planned Duration (weeks)

78

Number of Units

1

Contractor Rating (0-5)

4.2

Team Experience (years)

11

Number of Subcontractors

16

Number of Change Orders

5

Industry AI Flow

DashboardWorkflow ChatDocumentsDynamic AnalyticsCost EstimationIntent Demo

Demo UserLog out

DYNAMIC DATA ANALYSIS

AI-powered data understanding

Upload a dataset, ask a question — AI analyzes and visualizes in one step

Cloud LLM · Docker Sandbox · AI Charts

Step 1: Upload & Preview

Upload Data File (CSV / Excel)

Choose File No file chosen

Upload

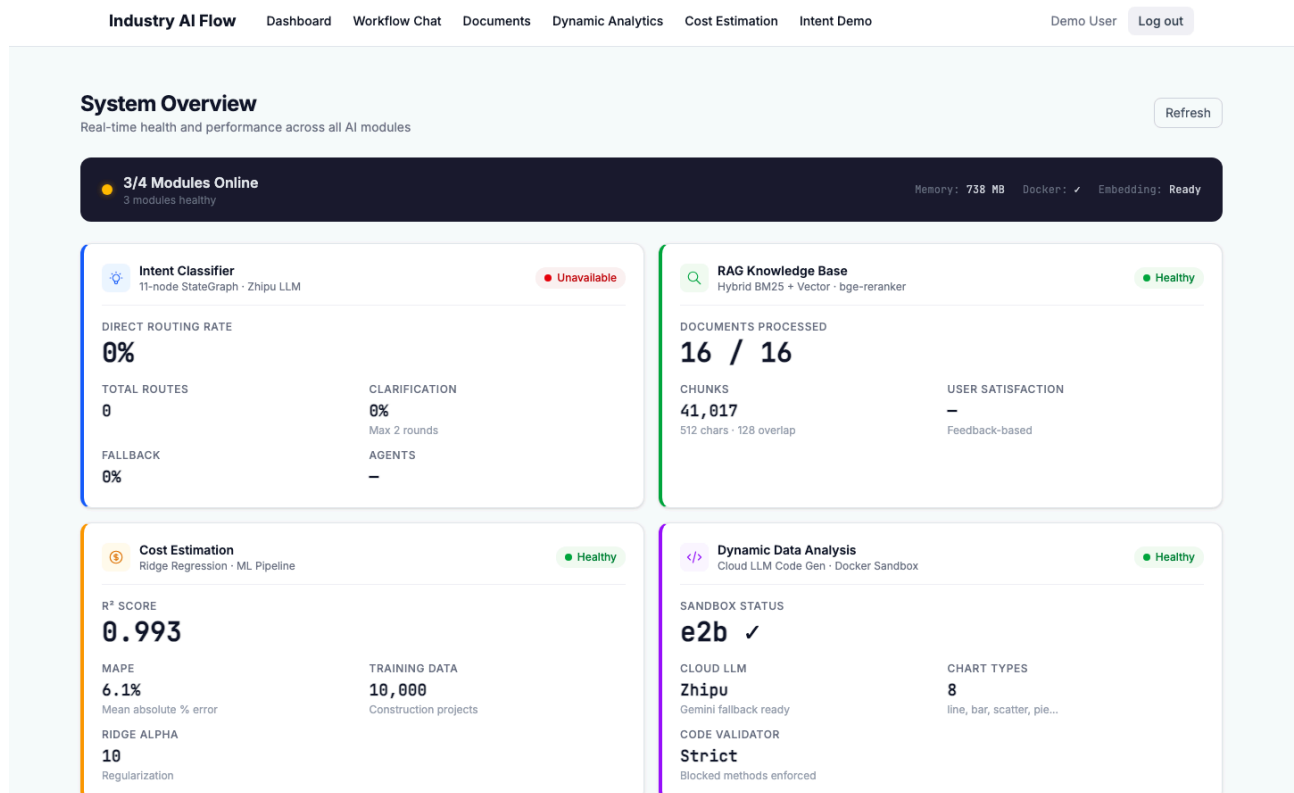
Use Sample (tips.csv)

Step 2: Ask & Analyze

Analysis Instruction

Ask anything about your data...

Run Analysis



8. Conclusion and Future Work

Summary

Industry AI Flow demonstrates that a small team can build a multi-capability AI system by focusing on architectural integration rather than individual model performance. The intent-aware routing architecture connects three distinct AI subsystems (RAG, ML prediction, and code generation) through a unified query interface, while maintaining source traceability, cost transparency, and data privacy.

The RAG system's hybrid retrieval pipeline (BM25 + vector + RRF + cross-encoder reranking) addresses the specific challenges of construction regulatory documents. The cost estimation module provides not just predictions but decision support tools (SHAP explainability, what-if analysis, similar project lookup). The dynamic analysis pipeline enables ad-hoc data exploration without exposing raw data to cloud LLMs.

Future Directions

- **Multi-tenant deployment:** The X-Tenant-ID isolation infrastructure is already built, supporting per-tenant rate limiting and audit logging for enterprise deployment
- **Prompt A/B testing:** A versioned prompt management system with performance scoring is in place, enabling systematic optimization of RAG response quality

- **RAG evaluation framework:** Developing ground truth relevance judgments for the 16-document corpus would enable quantitative evaluation of retrieval precision and recall
- **Model retraining:** As real (non-synthetic) construction cost data becomes available, retraining the CatBoost model on actual project outcomes would improve the overrun R^2 beyond the current 0.538

Reflections

The most valuable lesson from this project is that **the hard problems in applied AI are integration problems, not model problems**. Choosing between CatBoost and XGBoost matters far less than designing the intent classification pipeline that routes the right queries to the right subsystem. The 11-node StateGraph and the 10-node execution pipeline represent more engineering effort than any individual model — and they are the reason the system works as a coherent product rather than three disconnected demos.

References

- Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). Reciprocal rank fusion outperforms condorcet and individual rank learning methods. *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 758–759.
- Dorogush, A. V., Ershov, V., & Gulin, A. (2018). CatBoost: Gradient boosting with categorical features support. *arXiv preprint arXiv:1810.11363*.
- LangChain. (2024). LangGraph: Building stateful, multi-actor applications with LLMs. Retrieved from <https://langchain-ai.github.io/langgraph/>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.
- National Research Council Canada. (2020). *National Building Code of Canada 2020*. Ottawa: NRC.
- Statistics Canada. (2024). Building Construction Price Index (BCPI). Retrieved from <https://www150.statcan.gc.ca/t1/tbl1/en/tv.action?pid=1810013501>
- Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2024). C-Pack: Packaged resources to advance general Chinese embedding. *arXiv preprint arXiv:2309.07597*.
-

This article documents a Capstone project for the Integrated AI program at SAIT (Southern Alberta Institute of Technology) by Angel Bustamante, Jason Niu, and Jack Si. The live demo is available at <https://iai.dissidia.me/>. For questions, contact the team via SAIT.